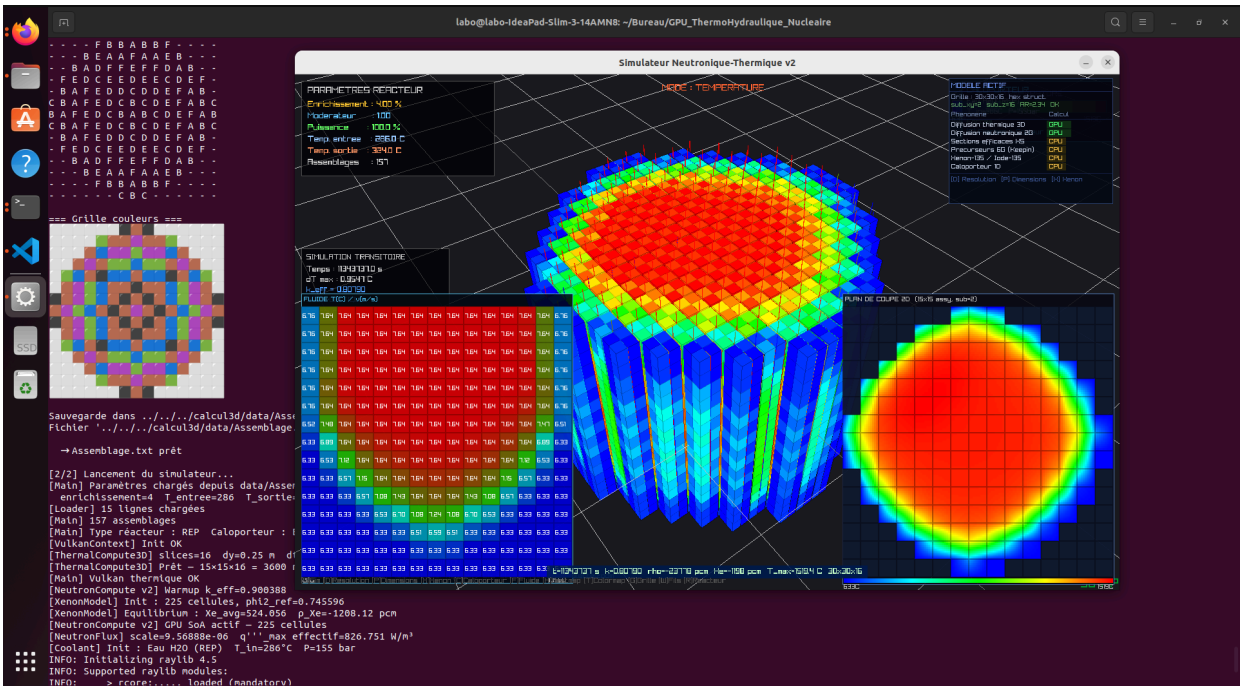
Eloi KRESS

VISUALISEUR THERMIQUE NUCLÉAIRE

Documentation Technique Complète

Version 2.0

Mars 2026



guide utilisateur partie 8, page 17

Table des matières

Table des matières....2
1. Introduction....3
2. Architecture générale du logiciel....5
2.1 Vue d’ensemble — pipeline en deux programmes....5
2.2 Solveur d’assemblage (C)....5
Placement symétrique d’ordre 8....6
Algorithmes d’optimisation Monte-Carlo....6
Format du fichier Assemblage.txt....6
2.3 Simulateur neutronique-thermique (C++)....6
2.4 Boucle principale de simulation....7
3. Fondements théoriques — Neutronique....8
3.1 Équation de diffusion neutronique à deux groupes d'énergie....8
3.2 Sections efficaces et données nucléaires....8
3.3 Discrétisation spatiale par différences finies....10
3.4 Itération de puissance et calcul de k_eff....10
3.5 Rétroaction Doppler....11
4. Fondements théoriques — Thermique....12
4.1 Équation de diffusion thermique 3D....12
4.3 Système de maillage adaptatif (MeshConfig)....12
4.4 Conditions aux limites....13
4.5 Couplage neutronique-thermique....13
5. Modèle caloporteur....14
Propriétés thermophysiques des fluides....14
Corrélations de Nusselt....14
6. Implémentation GPU — Pipeline Vulkan Compute....15
Architecture du pipeline thermique (diffusion.comp)....15
Architecture du pipeline neutronique v2 — SoA + FVM (neutron_fvm.comp + neutron_reduce.comp)....16
Fallback CPU....16
7. Guide d'installation et de compilation....17
7.1 Dépendances....17
7.2 Compilation sous Linux (Ubuntu 22.04 / Debian 12)....17
Étape 1 — Installation des dépendances système....17
Étape 2 — Compilation de Raylib depuis les sources....17
Étape 3 — Compilation des shaders GLSL → SPIR-V....18
Étape 4 — Compilation du binaire principal....18
7.3 Compilation sous Windows....18
Option A — Visual Studio 2022 (recommandé)....18
Option B — MinGW-w64 + MSYS2....19
8. Guide utilisateur....20
8.1 lancement du logiciel....20
8.1 Interface 3D et navigation....20
8.2 Panneaux de configuration....22
Panneau Réacteur [R]....22
Panneau Simulation [S]....24
Panneau Caloporteur [C]....25

8.3 Sélection du modèle de flux.....25
8.4 Interprétation des résultats.....26
9. Paramètres physiques par défaut.....27
10. Limites du modèle et perspectives.....28
Limites actuelles.....28
Perspectives d'évolution.....28

1. Introduction

Le présent document constitue la documentation technique de mon calculateur thermique, dédiés à la simulation temps réel des cœurs des réacteurs nucléaires. Il a été conçu pour permettre l'exploration interactive des phénomènes neutroniques et thermiques couplés, avec un rendu 3D immédiat des distributions de flux de température. Passionné par le nucléaire, l'objectif de la conception de ce logiciel est de synthétiser mes connaissances dans ce milieu et me rendre compte de mes limites dans la compréhension de cette discipline, d'une part, afin d'orienter mes recherches vers mes lacunes d'autre part.

L'objectif pratique final de la conception de ce logiciel est de connaître la température maximale en un élément du coeur en fonction de la disposition d'assemblage différents dans le coeur. J'ai pour vocation de coupler ce logiciel avec un autre de mes programmes qui utilise divers algorithmes comme Monte-Carlo, et même de l'IA afin de prédire des patternes de grilles d'assemblages homogènes.

Ce projet est effectué dans le cadre d'un projet d'année de mon école, à l'Ecole Centrale de Nantes. Il a pour but d'optimiser du calcul scientifique sur gpu. Ce logiciel est développé sur linux, mais devrait fonctionner sur windows, le code développé ne faisant pas appel à des fonctions OS.

Objectifs principaux :

- Simuler la distribution spatiale du flux neutronique dans un cœur de réacteur en utilisant la théorie de la diffusion à deux groupes d'énergie.
- Calculer en temps réel le facteur de multiplication effectif k_eff par itération de puissance.
- Résoudre l'équation de diffusion thermique 3D dans les assemblages combustibles par différences finies explicites sur GPU (Vulkan Compute).
- Modéliser le transfert de chaleur au caloporteur (eau, sodium, plomb-bismuth, hélium) par corrélations de Nusselt.
- Offrir une visualisation interactive 3D avec colormap de température et coupe 2D en plan.

Technologies de réacteurs supportées :

Technologie	Caloporteur/Modérateur	Enrichissement	T entrée typ.	Pression typ.
REP	Eau légère (H_2O)	3,5 %	286 °C	155 bar
CANDU	Eau lourde (D_2O)	0,7 % (naturel)	266 °C	100 bar
RNR-Na	Sodium liquide	15–20 %	370 °C	1 bar
RNR-Pb/Bi	Plomb-Bismuth	15–20 %	400 °C	1 bar

Technologie	Caloporteur/Modérateur	Enrichissement	T entrée typ.	Pression typ.
RHT	Hélium + Graphite	9 %	250 °C	70 bar
Sels fondus (thorium)	Fluorures fondus	1 %	565 °C	1 bar
UNGG	CO_2 + Graphite	0,3 %	250 °C	atm
RBMK	Eau légère (ébullition)	2 %	270 °C	65 bar

2. Architecture générale du logiciel

Le projet est composé de deux programmes distincts qui communiquent via un fichier texte intermédiaire, coordonnés par un script orchestrateur à la racine du projet.

2.1 Vue d'ensemble — pipeline en deux programmes

L'ensemble du projet repose sur deux programmes distincts qui s'exécutent séquentiellement. Le premier, écrit en C, configure le cœur de manière interactive et génère un fichier de configuration. Le second, écrit en C++, lit ce fichier et lance la simulation neutronique-thermique avec rendu GPU. Les deux sont coordonnés par un script orchestrateur à la racine du projet :

```
orchestrateur.sh  (racine du projet)
  ├─ [1/2]  assemblage_solver   (C, console, interactif)
  │         ↓ génère
  │    calcul3d/data/Assemblage.txt
  │         ↓ lu par
  └─ [2/2]  viewer_raylib        (C++, Vulkan/Raylib, GPU)
```

L'orchestrateur détecte automatiquement le système d'exploitation (Linux, macOS, Windows Git Bash / WSL) et lance les deux programmes en séquence. Si l'un des deux binaires est absent, il indique la commande de compilation à exécuter (make build). Si Assemblage.txt n'est pas généré en sortie du premier programme, l'orchestrateur s'arrête avec un message d'erreur explicite.

2.2 Solveur d'assemblage (C)

Le solveur d'assemblage est un programme en C (C99, console interactive) situé dans assemblage_solver/nuclear/code_c/. Il permet de configurer le cœur de manière interactive, d'optimiser la disposition des assemblages par des algorithmes stochastiques, et d'écrire le fichier Assemblage.txt lu par le simulateur.

Module	Fichiers principaux	Rôle
`Grid.c / .h`	Grille circulaire	Génération de la grille circulaire euclidienne (rayon paramétrable), correction des extrémités, allocation et libération mémoire.
`PlacementProgressif.c / .h`	Placement symétrique	Placement symétrique d'ordre 8 avec pénalité exponentielle sur l'usage des types. Garantit une distribution équilibrée et radialement symétrique.
`Thermique.c / .h`	Diffusion thermique 2D simplifiée	Diffusion thermique 2D par moyennage de voisinage (N itérations). Utilisée comme critère d'évaluation pendant l'optimisation Monte-Carlo. Critères disponibles : ΔT, gradient maximal, variance.
`MonteCarlo.c / .h`	Optimisation stochastique	Deux algorithmes : Metropolis (température fixe) et recuit simulé (décroissance T ← T × alpha). Minimisent le ΔT du cœur par échanges aléatoires d'assemblages.

Module	Fichiers principaux	Rôle
`ConfigReacteurs.c / .h`	Configurations standard	7 technologies préconfigurées (REP, CANDU, RNR-Na, RNR-Pb, Sels fondus, UNGG, RBMK) avec leurs paramètres thermiques et combustibles standards.
`TypesAssemblage.c / .h`	Saisie interactive et export	Définition interactive des types (symbole, puissance, combustible U235/U238/Pu239/MOX, enrichissement). Sauvegarde Assemblage.txt avec toutes les métadonnées.
`Affichage.c / .h`	Affichage terminal	Palette emoji (12 couleurs), sélection dynamique par type de symbole, rendu terminal de la grille colorée.

Placement symétrique d'ordre 8

L'algorithme parcourt un huitième du disque (i ≥ 0, j ≥ i) et applique les 8 symétries de chaque point autour du centre. La sélection du type d'assemblage utilise une pénalité exponentielle sur le nombre d'utilisations, garantissant une distribution équilibrée et radialement symétrique :

```
score(t) = exp(usage[t])
→ le type au score minimal est choisi → ses 8 positions symétriques reçoivent
ce type
```

Algorithmes d'optimisation Monte-Carlo

Deux algorithmes stochastiques optimisent la disposition des assemblages en minimisant le ΔT du cœur. À chaque itération, deux assemblages sont échangés aléatoirement dans le disque actif. L'échange est accepté ou refusé selon le critère de Metropolis :

```
Metropolis (température T fixe) :
  si ΔE < 0 : accepté    sinon : accepté avec proba p = exp(−ΔE / T)
Recuit simulé (alpha de décroissance) :
   même critère + T ← T × alpha   à chaque itération (exploration large →
convergence fine)
```

À chaque itération, 5 pas de diffusion thermique 2D simplifiée sont recalculés pour évaluer la qualité de la configuration candidate avant d'accepter ou rejeter l'échange.

Format du fichier Assemblage.txt

Ce fichier est l'unique interface de communication entre les deux programmes. Il est généré par sauver_assemblage() dans TypesAssemblage.c et lu par AssemblageLoader.cpp dans le simulateur. Il contient trois blocs :

```
# --- Métadonnées réacteur (lues par le simulateur C++) ---
# reacteur=REP        # enrichissement=4.00    # moderateur=1.00
# puissance=100.0     # tempEntree=286.0       # tempSortie=324.0
# caloporteur=EAU
# --- Grille (taille N, puis N lignes de N caractères) ---
11
----BCB----    ('-' = vide, lettre = type d'assemblage)
# --- Types (nb_types, puis 1 ligne par type) ---
A 1.000000 999 U235 4.000000 0.000000 0.000000 0.000000
```

2.3 Simulateur neutronique-thermique (C++)

Le simulateur est organisé en cinq répertoires fonctionnels qui communiquent via des structures de données partagées définies dans GridData.hpp.

Module	Fichiers principaux	Rôle
core/	GridData.hpp, ReactorParams.hpp	Structures de données : grille d'assemblages, paramètres physiques, zones neutroniques
physics/	NeutronFlux.hpp, NeutronCrossSection.hpp, CoolantModel.hpp, XenonModel.hpp, PrecursorData.hpp, ThermalModel.hpp	Calcul des sections efficaces, flux cosinus de référence, modèle fluide caloporteur
compute/	NeutronCompute.hpp, ThermalCompute.hpp, VulkanContext.hpp, VulkanCompute.hpp	Pipelines Vulkan Compute : diffusion neutronique 2G (CPU+GPU) et diffusion thermique 3D (GPU)
render/	CubeRenderer3D.hpp, SimPanel.hpp, ReactorPanel.hpp, CoolantPanel.hpp, XenonPanel.hpp, DimsPanel.hpp, MeshPanel.hpp, HeatmapPanel.hpp, ModeratorRenderer.hpp	Rendu 3D Raylib, panneaux ImGui-like, colormap, plan de coupe 2D
Shaders GLSL	diffusion.comp, neutron_fvm.comp, neutron_reduce.comp, shadow_rq.comp	Kernels Vulkan Compute : équations FDM compilées en SPIR-V

2.4 Boucle principale de simulation

À chaque image rendue, la boucle principale exécute la séquence suivante :

1. Mise à jour des sections efficaces : rebuildXS() recalcule les XS 2G de chaque cellule en fonction des températures courantes (rétroaction Doppler).
2. Pas neutronique : NeutronCompute::step() effectue N itérations du solveur FDM 2G (CPU ou CPU+GPU) et met à jour k_eff par itération de puissance.
3. Calcul de la puissance volumique : recalcFlux() convertit phi_total (flux normalisé [0,1]) en densité de puissance q'''(W/m³) en appliquant les constantes physiques et le facteur de puissance nominal.
4. Pas thermique : ThermalCompute::step() effectue stepsPerFrame itérations du shader de diffusion thermique sur GPU.
5. Mise à jour caloporteur : CoolantModel::update() calcule les températures de sortie par colonne d'assemblages.
6. Test de convergence : si maxΔT < seuil (0,05 °C par défaut), la simulation est arrêtée et le résultat affiché.
7. Rendu 3D : les températures sont projetées sur la colormap Jet et les assemblages sont dessinés.

3. Fondements théoriques — Neutronique

L'objectif des parties suivantes est de vous informer sur mes connaissances, afin que vous puissiez comprendre mes choix et me conseiller sur les améliorations, si vous le souhaitez.

3.1 Équation de diffusion neutronique à deux groupes d'énergie

La théorie de la diffusion neutronique à deux groupes est une approximation de l'équation de transport de Boltzmann. Elle regroupe le spectre énergétique des neutrons en deux domaines :

- **Groupe 1 (neutrons rapides) :** E > 0,625 eV — neutrons issus directement de la fission
- **Groupe 2 (neutrons thermiques) :** E < 0,625 eV — neutrons ayant atteint l'équilibre thermique avec le modérateur

Pour chaque groupe g ∈ {1, 2}, l'équation de diffusion en régime transitoire s'écrit :

Équation de diffusion 2G (forme différentielle)

$$1/vg \times \partial\varphi g/\partial t = \nabla\cdot(Dg\ \nabla\varphi g) - \Sigma Rg\ \varphi g + \Sigma S12\ \varphi 1\ [g=2] + (\chi g/k_eff)\ \Sigma g'(\nu\Sigma fg'\ \varphi g')$$

où les grandeurs sont :

Symbole	Unité	Signification physique
φg	$n\cdot cm^{-2}\cdot s^{-1}$	Flux neutronique du groupe g
vg	$cm\cdot s^{-1}$	Vitesse effective des neutrons du groupe g ($v_1 = 2\times10^7$ m/s, v_2 = 2200 m/s)
Dg	cm	Coefficient de diffusion : Dg = 1/(3 ΣTRg), homogénéisé sur la cellule
ΣRg	cm^{-1}	Section efficace macroscopique de retrait : absorption + transfert hors du groupe
ΣS12	cm^{-1}	Section de ralentissement groupe 1 → groupe 2 (modérateur)
νΣfg	cm^{-1}	Section de production par fission : ν × Σfg (nombre moyen de neutrons par fission × section de fission)
χg	sans dim.	Spectre de fission : fraction des neutrons nés dans le groupe g ($\chi_1 \approx 0{,}98$; $\chi_2 \approx 0{,}02$)
k_eff	sans dim.	Facteur de multiplication effectif : rapport neutrons produits / absorbés

3.2 Sections efficaces et données nucléaires

Les sections efficaces macroscopiques sont calculées à partir des bases de données microscopiques ENDF/B-VIII (https://www-nds.iaea.org/exfor/endf.htm) et homogénéisées sur la cellule combustible + modérateur selon la fraction volumique de combustible.

Homogénéisation cellulaire (REP) :

Pour un REP de pas de réseau p = 1,26 cm avec un crayon de rayon r = 0,41 cm :

- $V_{fuel} / V_{cellule} \approx \pi r^2 / p^2 \approx 0{,}30$ (fraction volumique combustible)
- $\nu\Sigma f_g$ hom. $= V_{fuel} \times N_{U235} \times \nu \times \sigma fg(U235) + V_{fuel} \times N_{U238} \times \nu \times \sigma fg(U238)$

Données nucléaires utilisées (U-235, spectre thermique) :

- $\sigma f(U235) = 585$ b, $\sigma a(U235) = 683$ b, $\nu = 2{,}43$
- $N_{UO2} = \rho NA/M = 10970 \times 6{,}022\times10^{23} / 270 = 2{,}32\times10^{22}$ at/cm³
- $\nu\Sigma f_{th}$ (REP 3,5%) $= 2{,}43 \times 8{,}1\times10^{20} \times 585\times10^{-24} \times 0{,}30 \approx 0{,}346$ cm⁻¹

Calibration des sections efficaces : *Pour obtenir k∞ ≈ 1,10 (REP frais, fuites ~8 %), la valeur effective homogénéisée est :*

Calibration nuSigF_th (REP 3,5%)
nuSigF[1] = 0,3230 × (ε/0,035) cm⁻¹ → k∞ = nuSigF[1]·ΣS12 / (ΣR1·ΣR2) = 1,10

Effet Doppler sur les sections efficaces :

L'élargissement résonnant des sections d'absorption de l'U-238 avec la température est modélisé par la correction :

- $dopp = 1 + 3{,}5\times10^{-3} \times (\sqrt{T} - \sqrt{T_{ref}}) \times (1 - \varepsilon)$
- T est la température du combustible en K, T_ref = 573,15 K (300 °C), ε est l'enrichissement en U-235.
- Cette correction augmente ΣR avec T, réduisant le flux et la puissance — c'est le coefficient de température Doppler négatif (rétroaction stabilisante).

3.3 Discrétisation spatiale par différences finies

Le domaine spatial est discrétisé sur une grille 2D régulière de maille dx × dz (un nœud par assemblage). Le laplacien est approché par le stencil centré à 5 points :

Laplacien FDM 5 points

```
∇·(D∇φ)_ij  ≈  [D_eff(i+1,j)·(φ(i+1,j)−φ(i,j))  +  D_eff(i−1,j)·(φ(i−1,j)−φ(i,j))
+ D_eff(i,j+1)·(φ(i,j+1)−φ(i,j)) + D_eff(i,j−1)·(φ(i,j−1)−φ(i,j))] / dx²
```

Le coefficient de diffusion à l'interface entre deux cellules est calculé par la moyenne harmonique, afin de conserver la continuité du courant neutronique :

- `D_eff(i→j) = 2·Di·Dj / (Di + Dj)`

Schéma semi-implicite : *Le terme d'absorption est traité implicitement pour garantir la stabilité quel que soit le pas de temps :*

Mise à jour semi-implicite (stable)

```
φg^(n+1) = [φg^n + dt·vg·(L_g + source_fission_g)] / (1 + dt·vg·ΣRg)
```

Critères de stabilité (pas de temps neutronique) :

- `Contrainte diffusion  : dt ≤ 0,45 × dx² / (4·D·v)`
- `Contrainte source  : dt ≤ 0,45 / (v₁ × νΣf_max)`
- `Valeur typique REP  : dt_neutron ≈ 1,3 s  (dx = 26 cm, D = 1,35 cm)`

Conditions aux limites :

- Bords externes du cœur : condition de Dirichlet φ = 0 (condition de vide extrapolé).
- Cellules de type VOID (interstices vides) : φ = 0.
- Zones modérateur : XS modérateur pur, nuSigF = 0.
- Zones réflecteur : XS acier + eau, nuSigF = 0, ΣR élevé.

3.4 Itération de puissance et calcul de k_eff

Le facteur de multiplication effectif k_eff est calculé à chaque pas de temps par itération de puissance. Cette méthode est standard en neutronique de réacteur.

Algorithme :

8. Calculer la production de fission avant le pas : $F^n = \Sigma_i(\nu\Sigma f1\cdot\varphi1_i + \nu\Sigma f2\cdot\varphi2_i)$
9. Effectuer un pas FDM semi-implicite -> obtenir φ^(n+1)
10. Calculer la nouvelle production : F^(n+1)
11. `k^(n+1) = k^n × F^(n+1) / F^n` (mise à jour proportionnelle)
12. Répéter 3 fois par step() pour accélérer la convergence.
13. Clamper k_eff ∈ [0,001 ; 10] pour éviter les divergences numériques.

Interprétation physique de k_eff :

k_eff	État du cœur	Signification
k_eff < 1	Sous-critique	Trop d'absorptions — le flux s'éteint spontanément
k_eff = 1	Critique	Régime stationnaire — production = absorption + fuites
k_eff > 1	Sur-critique	Population neutronique croissante — puissance en augmentation

3.5 Rétroaction Doppler

L'effet Doppler thermique est un phénomène rétroacdtif augmentant l'absorbtion de neutrons par les atomes en fonction de leur température . Lorsque la température du combustible augmente, l'élargissement des résonances d'absorption de l'U-238 augmente ΣR, ce qui réduit le flux neutronique et donc la puissance.

Le coefficient Doppler est défini par :

- $\alpha_D = (1/k) \times dk/dT_{fuel} \approx -3{,}0\times10^{-5}\ K^{-1}$ (REP typique)

Ce coefficient négatif garantit la stabilité intrinsèque : une augmentation de puissance échauffe le combustible, ce qui réduit k_eff, ce qui réduit la puissance. Ce mécanisme opère dans le logiciel via la mise à jour des XS à chaque pas de simulation.

4. Fondements théoriques — Thermique

4.1 Équation de diffusion thermique 3D

La distribution de température dans le combustible solide est gouvernée par l'équation de diffusion thermique avec terme source volumique :

Équation de diffusion thermique (forme différentielle)

$$\rho Cp \times \partial T/\partial t = \nabla \cdot (\lambda \nabla T) + q'''(x,y,z,t)$$

Où : $q'''(W/m^3) = \varphi_total \times \sigma f \times \varphi 0 \times E_fission \times 10^6 \times scale \times P_norm$

Propriétés thermophysiques de l'UO_2 utilisées :

Propriété	Valeur	Source
Conductivité thermique λ	3,6 W/(m·K)	IAEA-TECDOC-1236, T = 300 °C
Masse volumique ρ	10 970 kg/m³	UO_2 fritté, densité théorique 95 %
Capacité calorifique Cp	235 J/(kg·K)	Moyenne 300–1000 °C
Diffusivité thermique α = λ/(ρCp)	$1{,}39\times10^{-6}$ m²/s	Calculé
Température de fusion	1150 °C	Limite physique de clamp

Discrétisation 3D et pas de temps :

La grille thermique est constituée de cols × rows assemblages en plan (X,Z) et de nSlices tranches verticales (Y). L'objectif sera par la suite d'améliorer cette grille, mais pour l'heure, afin de ne pas trop me prendre la tête, j'ai fait ce choix exotique. Le pas de temps thermique est fixé par le critère de stabilité de Fourier 3D :

- $dt_th = 0{,}9 / (\alpha \times (4/dx^2 + 1/dy^2))$
- Valeur typique REP : dx = 0,26 m, 8 tranches → dy = 0,5 m → dt ≈ 5700 s

Note : ce pas de temps élevé est intrinsèque à l'UO_2 (faible diffusivité). L'option speedFactor permet d'accumuler plusieurs pas GPU par frame pour accélérer la convergence vers l'état stationnaire. Évidemment, c'est un des défauts car si l'on change de combustible, Mox, Thorium, on a probablement un comportement différents ; je suppose qu'un fonctionnement différent du coeur, comme avec la surgénération, induit un comportement différent.

4.3 Système de maillage adaptatif (MeshConfig)

À partir de la version 2, le maillage de calcul est entièrement découplé des dimensions physiques. La structure MeshConfig calcule automatiquement les dimensions de la grille à partir de trois paramètres géométriques et deux paramètres de subdivision :

Paramètre	Description
`assy_width_m`	Largeur nette d'un assemblage en mètres. Pitch = width + gap.
`assy_gap_m`	Jeu inter-assemblage en mètres. Sert aussi de diamètre hydraulique pour le caloporteur.

Paramètre	Description
`assy_height_m`	Hauteur active en mètres.
`sub_xy`	Divisions XZ par assemblage (1 à 65 536). cols = n_assy_cols × sub_xy. sub_xy=1 : 1 cellule/assemblage, sub_xy=4 : 16 cellules/assemblage.
`sub_z`	Divisions axiales Y (1 à 1 024). Contrôle la résolution verticale indépendamment de sub_xy.
`Cubes réguliers (auto)`	Calcule sub_z = round(hauteur / (largeur / sub_xy)) pour un rapport d'aspect ≈ 1.
`Mémoire GPU estimée`	Affichée en Mo/Go. Avertissement si >512 Mo ; indicateur « GPU REQUIS » si >4 Go.

Le module ThermalModel.hpp fournit deux utilitaires complémentaires : chargement de températures initiales depuis un fichier CSV (data/temperatures.csv, format row,col,temperature) pour démarrer la simulation depuis un état précalculé, et génération d'un profil gaussien simulé (centre chaud, bords froids) utile pour les tests sans simulation neutronique préalable.

4.4 Conditions aux limites

Frontière	Type	Valeur
Bords latéraux (X, Z)	Dirichlet	T = T_inlet (refroidi par caloporteur)
Base (Y = 0)	Dirichlet	T = T_inlet (entrée caloporteur froide)
Sommet (Y = H_actif)	Neumann	∂T/∂y = 0 (sortie libre, gradient nul)
Cellules vides (mask = 0)	Dirichlet	T = T_inlet (interstices refroidis)

4.5 Couplage neutronique-thermique

Le couplage entre les deux physiques est assuré à chaque pas de la boucle principale par le mécanisme suivant :

Neutronique → Thermique : le flux normalisé φ_total[i] ∈ [0,1] est converti en puissance volumique q'''[i] (W/m³) et uploadé dans le buffer GPU du shader thermique.

Thermique → Neutronique : les températures T[i] issues du shader thermique sont lues et appliquées au calcul Doppler dans rebuildXS(), modifiant les sections efficaces ΣR[i] = ΣR0 × dopp(T[i]).

Normalisation de la puissance volumique
`scale = (dT_cible × ρCp / dt_th) / (σf × φ0 × E_fission × 10⁶)q'''_eff[i] = φ_total[i] × σf × φ0 × E_fission × 10⁶ × scale × P_normP_norm = (Puissance%) / 100 × (ΔT_user / ΔT_ref)`

5. Modèle caloporteur

Dans cette partie j'aborde ce que j'ai déjà mis en place. J'ai par ailleurs de nombreux bugs à gérer. Par exemple, la simulation ne fonctionne pas si le temps n'est pas accéléré par deux. Si je divise un assemblage en 32 éléments, un comportement convergent si je le divise en moins d'éléments devient divergent.

Le module CoolantModel.hpp implémente un modèle de refroidissement colonne par colonne d'assemblages, avec calcul des propriétés thermophysiques du fluide en fonction de la température et de la pression.

Propriétés thermophysiques des fluides

Les propriétés sont calculées par corrélations polynomiales à partir des données IAPWS-97 (eau), données IAEA pour le sodium, et données IAEA-TECDOC pour le plomb-bismuth.

Fluide	ρ (kg/m³)	Cp (J/kg·K)	λ (W/m·K)	μ (mPa·s)	Pr
Eau (REP)	750	5 800	0,57	0,09	~0,9
Sodium (RNR)	850	1 260	65	0,28	0,004
Pb-Bi (RNR)	10 150	146,5	13	1,5	0,017
Hélium (RHT)	5,5	5 193	0,23	0,025	0,66

Corrélations de Nusselt

Le coefficient de transfert thermique h est calculé selon le régime d'écoulement :

- **Convection forcée turbulente (Re > 10 000) :** `Nu = 0,023 Re^0.8 Pr^n (Dittus-Boelter)`
- **Convection naturelle (Gr·Pr > 10^8) :** `Nu = 0,1 (Gr·Pr)^(1/3)`
- **Régime de transition :** *interpolation linéaire en log(Re)*

Bilan enthalpique par tranche :

- `T_fluid_out(z) = T_fluid_in(z) + q_assembly × dz / (ṁ × Cp)`
- `q_assembly = h × (T_combustible − T_fluide) × P_mouillé × dz`
- Le débit massique ṁ est calculé à partir de la vitesse du fluide, elle-même issue de la pression motrice de la pompe et des pertes de charge (corrélation Blasius).

6. Implémentation GPU — Pipeline Vulkan Compute

Le cœur de calcul haute performance est implémenté sous forme de compute shaders GLSL compilés en SPIR-V et exécutés via l'API Vulkan. Cette approche permet d'exploiter massivement le parallélisme du GPU pour les calculs FDM.

Architecture du pipeline thermique (diffusion.comp)

Le shader thermique traite en parallèle toutes les cellules de la grille 3D (cols × rows × slices). Chaque thread GPU est responsable d'une tranche d'un assemblage. Un shader est en langage Vulkan une fonction qui va être exécutée sur le gpu. (kernel en langage Cuda / AMD).

Binding	Type	Contenu
0	SSBO	T_in[cols×rows×slices] : températures au pas n (float)
1	SSBO	T_out[cols×rows×slices] : températures au pas n+1 (float)
2	SSBO	q_vol[cols×rows] : puissance volumique 2D (float, W/m³)
3	SSBO	mask[cols×rows] : masque de cellules actives (int, 0=vide)
4	UBO	SimParams : α, dx, dt, ρCp, T_inlet, T_max, dimensions

La stratégie ping-pong alterne les rôles de T_in et T_out à chaque itération sans copie mémoire. Le groupe de travail est configuré à 8×8×4 threads pour correspondre aux lignes de cache GPU.

notes :

- *Dans un calcul parallèle sur GPU, la stratégie ping-pong consiste à utiliser deux tampons mémoire, notés Tin et Tout, et à **échanger leurs rôles à chaque itération**. À l'itération n, le shader lit dans Tin et écrit dans Tout ; à l'itération n+1, les deux pointeurs sont intervertis. Cette méthode évite toute **copie mémoire globale**, ce qui réduit fortement les transferts et améliore l'efficacité du calcul. C'est d'autant plus important que les transfert mémoires sont très lent vis à vis du temps d'exécution sur une unité de calcul.*
- *Un **groupe de travail** (aussi workgroup) est un bloc de threads qui coopèrent pour exécuter une partie d'un calcul parallèle sur GPU. Il optimise l'accès mémoire, la synchronisation et l'utilisation des unités de calcul. Sa taille est choisie pour correspondre aux contraintes matérielles du GPU et à la structure du problème (2D, 3D, stencil, diffusion, etc.).*
- *Un **thread**, un **workgroup**, un **warp**, et un **dispatch** sont quatre niveaux d'organisation du calcul parallèle sur GPU. Ils forment une hiérarchie qui explique comment un shader compute est réellement exécuté. Les distinguer clairement permet de comprendre pourquoi on choisit des tailles comme 8×8×4 et comment un GPU exploite sa mémoire et ses unités SIMD.*
- *À l'intérieur d'un groupe de travail, les threads ne sont pas exécutés individuellement mais par paquets matériels appelés wavefronts (AMD) ou warps (NVIDIA). Un wavefront regroupe 64 threads qui avancent de manière strictement synchrone : ils exécutent la même instruction au même cycle, dans un mode SIMD. Cette contrainte impose que les threads d'un même wavefront accèdent à des données contiguës et suivent un flot de contrôle identique, faute de quoi l'exécution se séquentialise et les performances chutent. La taille fixe de 64 threads par wavefront influence donc directement le choix de la taille d'un groupe de travail : celui-ci doit être un multiple de 64 pour garantir une occupation complète des unités SIMD et éviter que des threads restent inactifs.*
-

Architecture du pipeline neutronique v2 — SoA + FVM (neutron_fvm.comp + neutron_reduce.comp)

À partir de la version 2, le solveur neutronique utilise une architecture SoA (Structure of Arrays) et deux shaders distincts : neutron_fvm.comp résout les équations FDM 2G nœud par nœud, et neutron_reduce.comp calcule la production de fission totale sur GPU pour la mise à jour de k_eff sans transfert CPU. Les deux groupes φ1 et φ2 sont mis à jour simultanément.

Binding	Type	Contenu
0–3	SSBO	phi1_in, phi2_in, phi1_out, phi2_out (ping-pong 2G, architecture SoA)
4	SSBO	xs_buf[total2d × 9] : 9 XS par cellule (D1,D2,ΣR1,ΣR2,ΣS12,νΣf1,νΣf2,χ1,χ2)
5	SSBO	zone_buf[total2d] : type de zone (0=void, 1=fuel, 2=mod, 3=réfl, 4=barre)
6	UBO	NeutronParams : dt, dx, k_eff, grid_cols, grid_rows, v1, v2

Fallback CPU

Si le shader SPIR-V neutron_fvm.spv n'est pas disponible (fichier absent ou Vulkan non supporté), le calcul neutronique bascule automatiquement sur l'implémentation CPU qui implémente exactement le même algorithme FDM. L'indicateur gpuAccel = false est alors affiché dans le panneau [S]. Cependant, l'efficacité du gpu dans des calculs parallèle risque de rendre le cpu inopérant s'il devait reprendre la main.

7. Guide d'installation et de compilation

Afin de faciliter la compilation, j'ai généré un petit script à l'aide de l'ia afin de compiler plus facilement, et de ne pas polluer le terminal avec des messages inutiles. Ce dernier fonctionne sur linux, mais d'autre sont rédigés ci-dessous pour un fonctionnement sur windows.

7.1 Dépendances

Bibliothèque	Version min.	Rôle	Site officiel
Raylib	5.0	Fenêtrage, rendu 3D, entrées	raylib.com
Vulkan SDK	1.3	Compute shaders GPU	vulkan.lunarg.com
glslangValidator	14.x (incl. SDK)	Compilation GLSL → SPIR-V	Inclus dans Vulkan SDK
GCC / G++	11+	Compilation C++17	gcc.gnu.org
MSVC	2022	Compilation Windows	visualstudio.microsoft.com
libX11, libGL	Système	Contexte graphique Linux	Paquets système

7.2 Compilation sous Linux (Ubuntu 22.04 / Debian 12)

Étape 1 — Installation des dépendances système

```
# Mise à jour du système
sudo apt update && sudo apt upgrade -y

# Compilateur C++17
sudo apt install -y g++ build-essential cmake

# Bibliothèques graphiques système
sudo apt install -y libx11-dev libxrandr-dev libxinerama-dev \
                    libxcursor-dev libxi-dev libgl1-mesa-dev

# Vulkan SDK (méthode officielle LunarG)
wget -qO- https://packages.lunarg.com/lunarg-signing-key-pub.asc | sudo tee \
    /etc/apt/trusted.gpg.d/lunarg.asc
sudo wget -qO /etc/apt/sources.list.d/lunarg-vulkan-jammy.list \
    https://packages.lunarg.com/vulkan/lunarg-vulkan-jammy.list
sudo apt update
sudo apt install -y vulkan-sdk

# Vérification Vulkan
vulkaninfo --summary
```

Étape 2 — Compilation de Raylib depuis les sources

```
# Cloner Raylib 5.0
git clone --depth 1 --branch 5.0.0 https://github.com/raysan5/raylib.git
cd raylib/src

# Compilation en bibliothèque statique
make PLATFORM=PLATFORM_DESKTOP

# Installation système
```

```
sudo cp libraylib.a /usr/local/lib/
sudo cp raylib.h   /usr/local/include/
cd ../..
```

Étape 3 — Compilation des shaders GLSL → SPIR-V

```
# Depuis le répertoire racine du projet
glslangValidator -V --target-env vulkan1.3 \
    compute/shaders/diffusion.comp \
    -o compute/shaders/diffusion.spv

glslangValidator -V --target-env vulkan1.3 \
    compute/shaders/neutron_fvm.comp \
    -o compute/shaders/neutron_fvm.spv
glslangValidator -V --target-env vulkan1.3 \
    compute/shaders/neutron_reduce.comp \
    -o compute/shaders/neutron_reduce.spv

# Vérification
ls -lh compute/shaders/*.spv
# Attendu : diffusion.spv, neutron_fvm.spv et neutron_reduce.spv (~2-5 Ko chacun)
```

Étape 4 — Compilation du binaire principal

```
# Compilation complète
g++ main.cpp core/AssemblageLoader.cpp \
    -I. \
    -o viewer_raylib \
    -lraylib -lvulkan -lGL -lm -lpthread -ldl -lrt -lX11 \
    -std=c++17 -O2

# Ou via le script fourni
chmod +x build.sh
./build.sh

# Lancement
./viewer_raylib
```

Script build.sh fourni : *Le script compile automatiquement les shaders puis le C++ en une seule commande.*

7.3 Compilation sous Windows

Option A — Visual Studio 2022 (recommandé)

```
:: 1. Installer le Vulkan SDK depuis https://vulkan.lunarg.com/sdk/home
::    (ajoute glslangValidator.exe au PATH automatiquement)

:: 2. Installer Raylib via vcpkg
git clone https://github.com/microsoft/vcpkg.git
cd vcpkg
.\bootstrap-vcpkg.bat
.\vcpkg install raylib:x64-windows

:: 3. Compiler les shaders (PowerShell ou cmd)
glslangValidator.exe -V --target-env vulkan1.3 ^
    compute\shaders\diffusion.comp ^
    -o compute\shaders\diffusion.spv

glslangValidator.exe -V --target-env vulkan1.3 ^
    compute\shaders\neutron_fvm.comp ^
    -o compute\shaders\neutron_fvm.spv

:: 4. Créer un projet Visual Studio et ajouter :
```

```
::     - Inclure : $(VULKAN_SDK)\Include ; chemin\raylib\include ; .
::     - Libs    : $(VULKAN_SDK)\Lib\vulkan-1.lib ; raylib.lib ; opengl32.lib
::     - Standard C++ : /std:c++17
```

Option B — MinGW-w64 + MSYS2

```
# Installer MSYS2 depuis https://www.msys2.org

# Depuis MSYS2 MinGW64 terminal
pacman -S mingw-w64-x86_64-gcc mingw-w64-x86_64-raylib

# Shaders (même commande que Linux)
glslangValidator -V --target-env vulkan1.3 \
    compute/shaders/diffusion.comp -o compute/shaders/diffusion.spv

# Compilation
g++ main.cpp core/AssemblageLoader.cpp -I. \
    -o viewer_raylib.exe \
    -lraylib -lvulkan -lopengl32 -lgdi32 -lwinmm \
    -std=c++17 -O2
```

Note sur le support GPU Windows

Le Vulkan SDK sous Windows installe automatiquement les drivers de validation. Si votre GPU ne supporte pas Vulkan 1.3, le logiciel bascule automatiquement sur le solveur CPU pour la neutronique. Le shader thermique (diffusion.spv) requiert cependant Vulkan. En cas d'absence de GPU compatible, une version de secours utilisant le CPU pur est envisageable comme extension.

8. Guide utilisateur

8.1 lancement du logiciel

La compilation peut prendre plusieurs seconde ; le compilateur doit compiler le solveur (langage c), et le viewer composé de fichiers c++ et de shader à compiler séparément.

Compiler le projet uniquement (make build) :

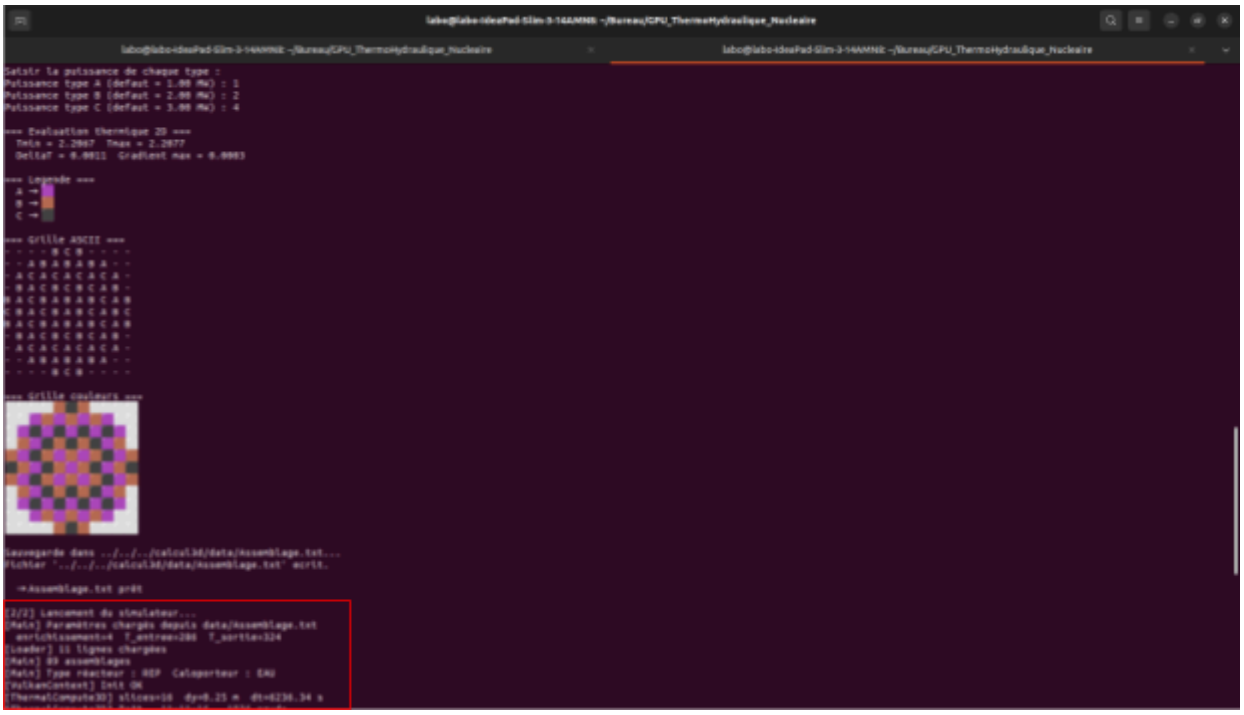
```
labo@labo-IdeaPad-Slim-3-14AMN8:~/Bureau/GPU_ThermoHydraulique_Nucleaire$ make build
=== Initialisation submodules ===
git submodule update --init --recursive
OK
=== Compilation solveur (C) ===
=== Compilation solveur assemblage ===
Sources : ./main.c
./PlacementProgressif.c
./Affichage.c
./TypesAssemblage.c
./ConfigReacteurs.c
./MonteCarlo.c
./Thermique.c
./Grid.c
In file included from ./TypesAssemblage.c:5:
./ConfigReacteurs.h:20:20: warning: 'NOMS_REACTEURS' defined but not used [-Wunused-variable]
   20 | static const char *NOMS_REACTEURS[] = {
      |                    ^~~~~~~~~~~~~~
In file included from ./ConfigReacteurs.c:1:
./ConfigReacteurs.h:20:20: warning: 'NOMS_REACTEURS' defined but not used [-Wunused-variable]
   20 | static const char *NOMS_REACTEURS[] = {
      |                    ^~~~~~~~~~~~~~
=== OK → /home/labo/Bureau/GPU_ThermoHydraulique_Nucleaire/assemblage_solver/nuclear/code_c/assemblage_solver ===
=== Compilation simulateur (C++) ===
=== Compilation shaders SPIR-V ===
  compute/shaders/diffusion.comp → compute/shaders/diffusion.spv
compute/shaders/diffusion.comp
  compute/shaders/neutron_fvm.comp → compute/shaders/neutron_fvm.spv
compute/shaders/neutron_fvm.comp
  compute/shaders/neutron_reduce.comp → compute/shaders/neutron_reduce.spv
compute/shaders/neutron_reduce.comp
=== Shaders OK ===

=== Compilation C++ ===
Sources :
./main.cpp
./core/AssemblageLoader.cpp
=== Build OK → ./viewer_raylib ===

=== Build terminé ===
Lance : make run
```

compiler le projet puis le lancer dans la foulée :

```
labo@labo-IdeaPad-Slim-3-14AMN8: ~/Bureau/GPU_ThermoHydraulique_Nucleaire
labo@labo-IdeaPad-Slim-3-14AMN8: ~/Bureau/GPU_ThermoHydraulique_Nucleaire   labo@labo-IdeaPad-Slim-3-14AMN8: ~/Bureau/GPU_ThermoHydraulique_Nucleaire
labo@labo-IdeaPad-Slim-3-14AMN8:~/Bureau/GPU_ThermoHydraulique_Nucleaire$ make run
=== Initialisation submodules ===
git submodule update --init --recursive
OK
=== Compilation solveur (C) ===
=== Compilation solveur assemblage ===
Sources : ./main.c
./PlacementProgressif.c
./Affichage.c
./TypesAssemblage.c
./ConfigReacteurs.c
./MonteCarlo.c
./Thermique.c
./Grid.c
In file included from ./TypesAssemblage.c:5:
./ConfigReacteurs.h:20:20: warning: 'NOMS_REACTEURS' defined but not used [-Wunused-variable]
   20 | static const char *NOMS_REACTEURS[] = {
      |                    ^~~~~~~~~~~~~~
In file included from ./ConfigReacteurs.c:1:
./ConfigReacteurs.h:20:20: warning: 'NOMS_REACTEURS' defined but not used [-Wunused-variable]
   20 | static const char *NOMS_REACTEURS[] = {
      |                    ^~~~~~~~~~~~~~
=== OK → /home/labo/Bureau/GPU_ThermoHydraulique_Nucleaire/assemblage_solver/nuclear/code_c/assemblage_solver ===
=== Compilation simulateur (C++) ===
=== Compilation shaders SPIR-V ===
  compute/shaders/diffusion.comp → compute/shaders/diffusion.spv
compute/shaders/diffusion.comp
  compute/shaders/neutron_fvm.comp → compute/shaders/neutron_fvm.spv
compute/shaders/neutron_fvm.comp
  compute/shaders/neutron_reduce.comp → compute/shaders/neutron_reduce.spv
compute/shaders/neutron_reduce.comp
=== Shaders OK ===

=== Compilation C++ ===
Sources :
./main.cpp
./core/AssemblageLoader.cpp
=== Build OK → ./viewer_raylib ===

=== Build terminé ===
Lance : make run
==========================================
  Simulateur Neutronique-Thermique
  Système détecté : linux
==========================================

[1/2] Solveur d'assemblage...
      (réponds aux questions pour configurer le cœur)

Mode d'utilisation :
 1) Standard (reacteur preconfigure)
 2) Personnalise (tout saisir)
Votre choix :
```

Pour une première utilisation, il est conseillé de sélectionner le mode standard et de saisir les valeurs recommandées. voici des captures d'écrans illustrant les différentes saisies après l'exécution de `make run`.

```
labo@labo-IdeaPad-Slim-3-14AMN8: ~/Bureau/GPU_ThermoHydraulique_Nucleaire
labo@labo-IdeaPad-Slim-3-14AMN8: ~/Bureau/GPU_ThermoHydraulique_Nucleaire   labo@labo-IdeaPad-Slim-3-14AMN8: ~/Bureau/GPU_ThermoHydraulique_Nucleaire

=== Build terminé ===
Lance : make run
==========================================
  Simulateur Neutronique-Thermique
  Système détecté : linux
==========================================

[1/2] Solveur d'assemblage...
      (réponds aux questions pour configurer le cœur)

Mode d'utilisation :
 1) Standard (reacteur preconfigure)
 2) Personnalise (tout saisir)
Votre choix : 1
Rayon du coeur (defaut 5) : 5

=== Grille circulaire (taille = 11) ===
. . . . 0 0 0 . . . .
. . 0 0 0 0 0 0 0 . .
. 0 0 0 0 0 0 0 0 0 .
. 0 0 0 0 0 0 0 0 0 .
0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0
. 0 0 0 0 0 0 0 0 0 .
. 0 0 0 0 0 0 0 0 0 .
. . 0 0 0 0 0 0 0 . .
. . . . 0 0 0 . . . .

Choisissez un modele standard :
 1) REP
 2) CANDU
 3) RNR sodium
 4) RNR plomb
 5) Sels fondus (thorium)
 6) UNGG
 7) RBMK
Votre choix : 1

Configuration REP chargee (3 types).
  Caloporteur : Eau (H2O)
  T entree : 286°C  T sortie : 324°C
  Moderateur : 1.0

Nombre de types souhaite (defaut = 3) : 3

Saisir la puissance de chaque type :
Puissance type A (defaut = 1.00 MW) : 1
Puissance type B (defaut = 2.00 MW) : 2
Puissance type C (defaut = 3.00 MW) : 4

=== Evaluation thermique 2D ===
```

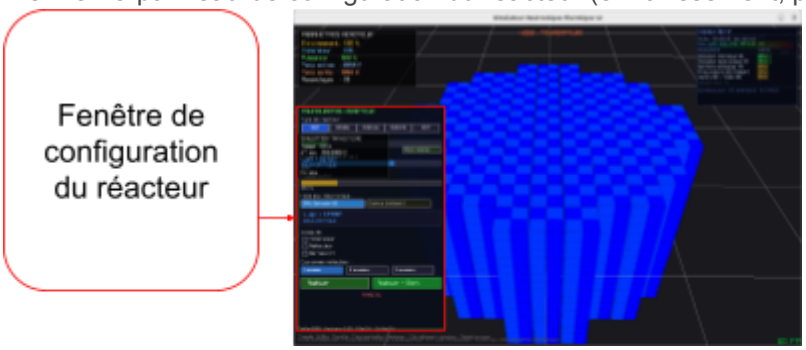


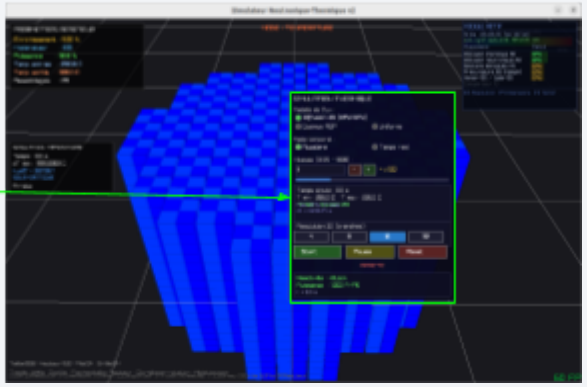
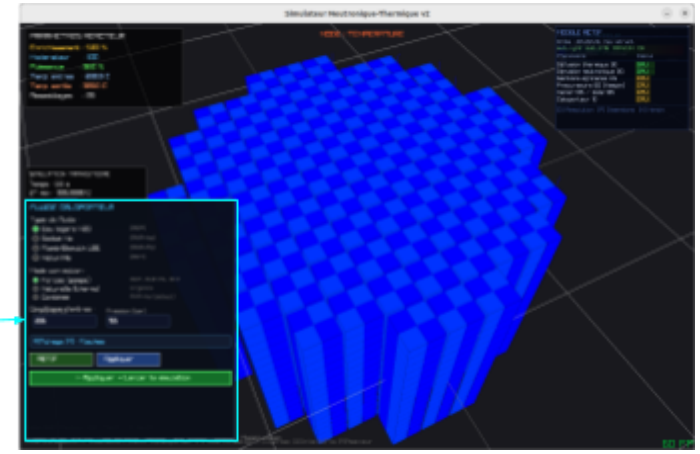
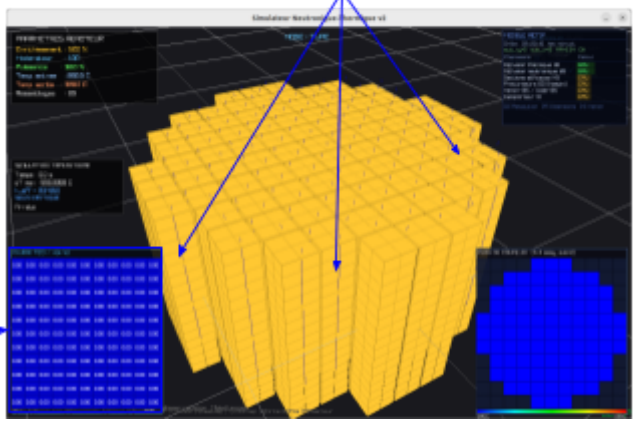
Comme encadré en rouge, le programme de calcul et visualisation 3D se lance automatiquement.

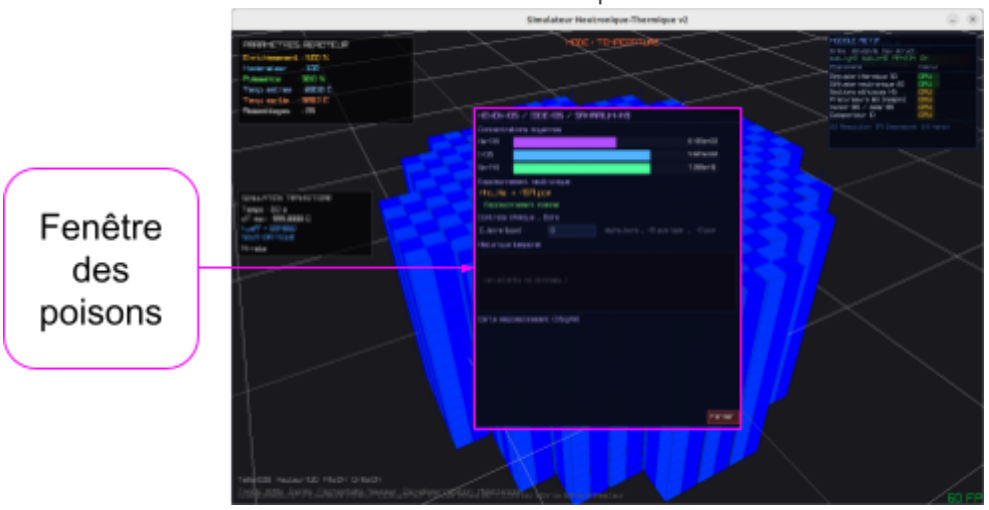
8.1 Interface 3D et navigation

Au lancement, le logiciel affiche le cœur du réacteur en 3D avec les assemblages combustibles représentés sous forme de prismés hexagonaux ou carrés selon la géométrie chargée.

Contrôles clavier :

Touche	Action
R	Ouvrir/fermer le panneau de configuration du réacteur (enrichissement, puissance,  type)
S	Ouvrir/fermer le panneau de simulation (modèle de flux, vitesse, lancer/arrêter)

	Fenêtre de configuration de la simulation 
C	Ouvrir/fermer le panneau caloporteur (type fluide, T_entrée, pression) Fenêtre de sélection du fluide caloporteur 
H	Ouvrir/fermer le panneau de coupe 2D (plan de coupe horizontal)
T	Basculer entre mode colormap température et mode couleur par type de zone
G	Afficher/masquer la grille de repère
F	Basculer entre vue colormap fluide et représentation fléché 3D (nécessite un caloporteur actif. trois mode : colormap, représentation fléché 3D, les deux à la fois) traits bleu représentant la circulation du fluide (de bas en haut) colormap représentant la température du fluide en fonction de sa position 

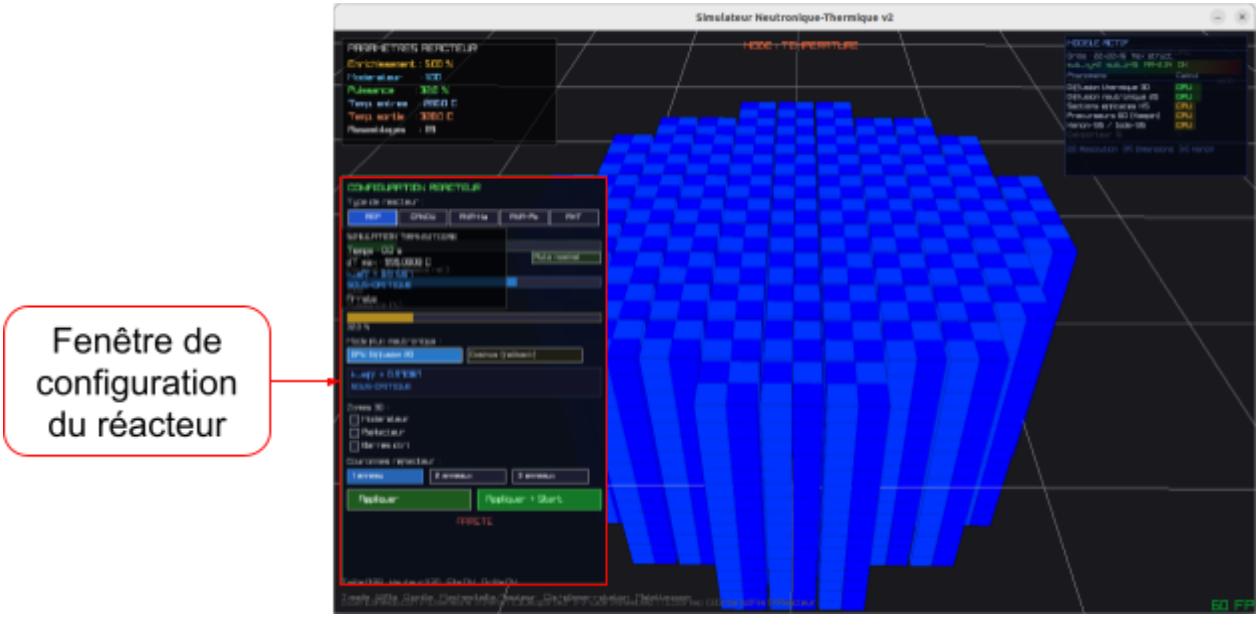
X	Afficher la fenêtre des poisons du coeurs 
Molette	Zoom avant / arrière
Clic droit + glisser	Rotation de la vue 3D
Flèches	Déplacement du plan de coupe horizontal
Z	Afficher/masquer les fils de grille sur les assemblages

Interface visuelle :

- La barre de légende colormap en haut à droite indique T_min (bleu) et T_max (rouge) de la simulation courante.
- Le panneau de simulation en haut à gauche affiche en temps réel : temps simulé, dT_max par frame, k_eff, état CRITIQUE/SOUS-CRITIQUE/SUR-CRITIQUE, et état de CONVERGENCE.
- Le plan de coupe 2D (panneau H ou bas-droit) affiche une vue en plan de la distribution de température à une tranche verticale sélectionnable.
- Le bandeau permanent en bas de l'écran affiche en temps réel, pendant toute la simulation : t= (temps simulé en secondes), k= (k_eff sur 5 décimales), rho= (réactivité en pcm, préfixé + ou −), Xe= (empoisonnement xénon en pcm), T_max= (température maximale du combustible en °C) et les dimensions du maillage courant (cols×rows×slices). La couleur du fond change selon la criticité : vert foncé (critique), orange (surgénérant), bleu foncé (sous-critique).
- Un bandeau vert en bas indique la convergence : « CONVERGE à t=7438 s T_max=380,0 °C [Diffusion 2G] ».

8.2 Panneaux de configuration

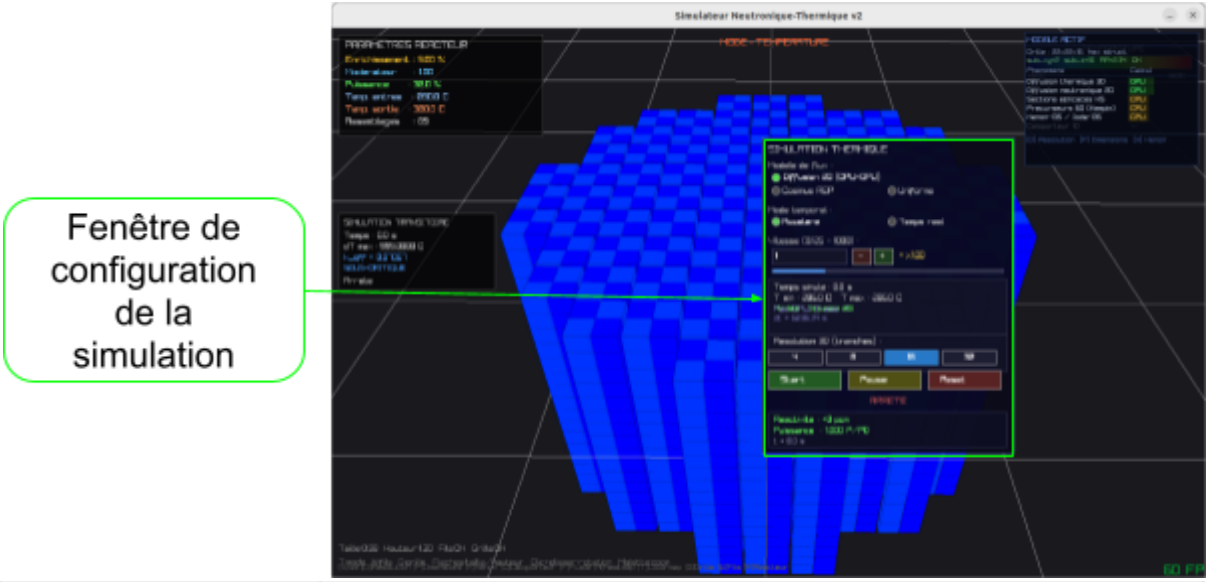
Panneau Réacteur [R]



Ce panneau permet de configurer la physique du cœur avant (ou pendant) la simulation.

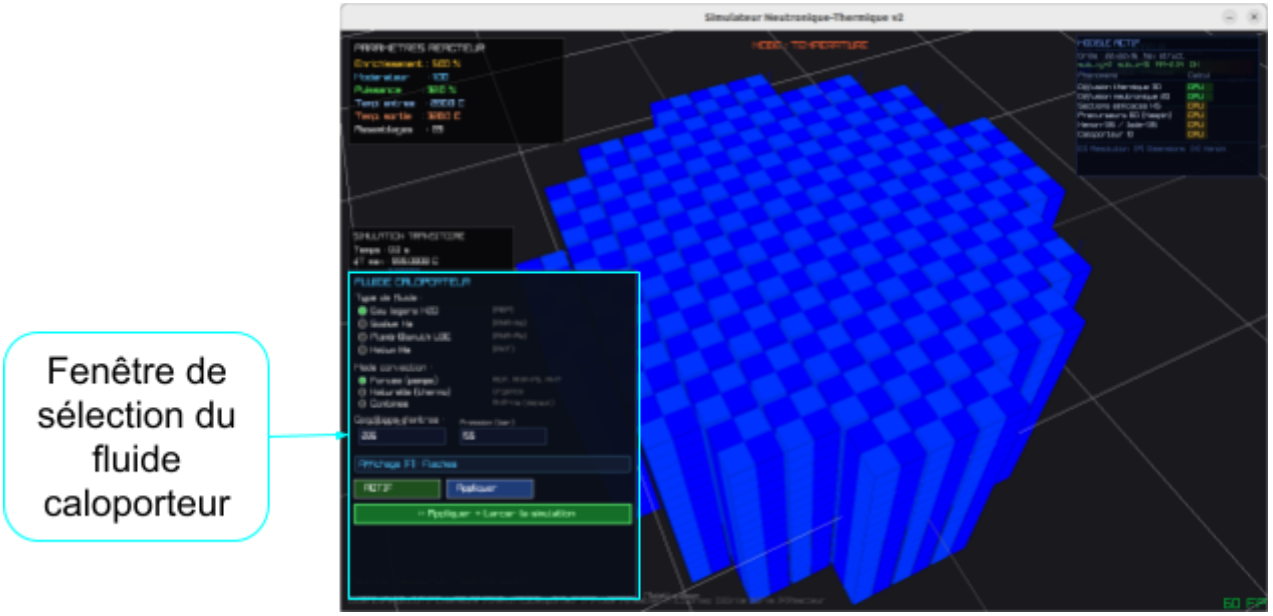
Paramètre	Description et plage
Type de réacteur	Choix entre REP, CANDU, RNR-Na, RNR-Pb, RHT. Modifie toutes les sections efficaces et les propriétés du caloporteur.
Enrichissement ε	[0,5 % – 25 %] — fraction massique U-235. Impact direct sur nuSigF et k_eff.
Densité modérateur	[0,1 – 1,5] — densité relative du modérateur (1,0 = nominal). Permet de simuler un vide caloporteur partiel.
Puissance nominale	[5 % – 100 %] — fraction de puissance par rapport au nominal. Échelle directement q'''.
T_entrée / T_sortie	Températures de référence du caloporteur (°C). Utilisées pour la normalisation de q'''.
Bouton "Appliquer"	Recalcule les XS, réinitialise le flux neutronique, sans relancer la simulation.
Bouton "Appliquer + Start"	Applique les paramètres ET lance immédiatement la simulation.
Mode flux neutronique	Bascule entre GPU Diffusion 2G (solveur physique complet) et Cosinus REP (profil analytique de repli). Deux boutons dans le panneau.
k_eff affiché	Indicateur coloré en temps réel : vert (critique ±0,5 %), rouge (surgénérant), bleu (sous-critique). Affiché en permanence dans le panneau.
Zones 3D (cases à cocher)	Afficher/masquer en 3D le modérateur, le réflecteur et les barres de contrôle (cylindres B4C animés).
Couronnes réflecteur	Sélection du nombre d'anneaux réflecteurs autour du cœur actif : 1, 2 ou 3. Impact direct sur les conditions aux limites neutroniques.

Panneau Simulation [S]



Paramètre	Description
Modèle de flux	Trois options : Diffusion 2G (modèle physique complet, recommandé), Cosinus REP (profil analytique approximatif), Uniforme (source homogène).
Mode temporel	Accéléré : plusieurs pas GPU par frame (recommandé pour convergence rapide). Temps réel : 1 pas par frame.
Vitesse (speedFactor)	[0,125 – 1000] — multiplicateur du nombre de pas thermiques par frame. Valeur > 1 : stepsPerFrame = speedFactor. Valeur < 1 : 1 pas toutes les 1/speedFactor frames.
Start / Pause / Stop	Contrôle de l'état de la simulation. Stop remet T = T_inlet et réinitialise le flux.
Nombre de tranches	[4, 8, 16, 32] — discrétisation verticale. Plus de tranches = meilleure résolution axiale mais dt plus petit.
Courbes temporelles	Graphes k_eff(t), P(t) et T_max(t) affichés dans le panneau. Historique sur 5 000 points. Permet de visualiser les transitoires et la convergence.
Réactivité (pcm)	$\rho = (k_eff - 1) / k_eff \times 10^5$ affichée en pcm en temps réel dans le panneau et dans le bandeau bas de l'écran.

Panneau Caloporteur [C]



Paramètre	Description
Fluide	Eau, Sodium, Plomb-Bismuth, Hélium — change les corrélations thermophysiques et la pression nominale.
T_inlet	Température d'entrée du caloporteur (bas du cœur) en °C.
Pression	Pression du circuit primaire en bar. Affecte le point d'ébullition de l'eau (REP).
Mode convection	Forcée (REP, RNR), Naturelle (convection libre), Combinée (mixte).
Actif / Inactif	Active ou désactive le refroidissement par caloporteur. Si inactif, conditions aux limites Dirichlet pures.
Bouton "Appliquer + Lancer"	Active le caloporteur, applique les paramètres et lance la simulation.

8.3 Sélection du modèle de flux

Le choix du modèle de flux est l'un des paramètres les plus importants pour la qualité des résultats. Il est accessible depuis le panneau [S].

Modèle Diffusion 2G (recommandé)

Résout les équations de diffusion neutronique à deux groupes d'énergie par différences finies (CPU, GPU optionnel). Calcule k_eff dynamiquement. Tient compte des zones combustible, modérateur, réflecteur, barres de contrôle. Rétroaction Doppler active. Affiché en vert dans le panneau si disponible.

Modèle Cosinus REP

Profil analytique $\varphi(r) = \varphi_0 \times \cos(\pi \cdot r / 2R)$, où R est le rayon du cœur. Plus rapide mais sans calcul de k_eff ni rétroaction. Utile pour des vérifications rapides ou lorsque le solveur 2G n'est pas disponible. Inclut la rétroaction Doppler via le coefficient α_D.

Comportement au lancement : si NeutronCompute::init() réussit (CPU disponible), le modèle Diffusion 2G est sélectionné automatiquement. Si la neutronique CPU échoue (mémoire insuffisante), le logiciel bascule sur Cosinus REP sans interruption.

8.4 Interprétation des résultats

Indicateur k_eff :

- k_eff ≈ 0,999–1,001 : cœur critique → régime stationnaire stable. Le flux se stabilise à une distribution cosinus déformée par les XS hétérogènes.
- k_eff < 0,99 : cœur sous-critique. Le flux s'amortit exponentiellement. Peut indiquer un enrichissement trop faible ou des barres de contrôle trop insérées.
- k_eff > 1,01 : cœur sur-critique. La puissance et la température augmentent. Le feedback Doppler finit par ramener k_eff vers 1 si α_D est négatif.

Convergence thermique :

- La convergence est atteinte lorsque le changement de température maximal entre deux frames (maxΔT) est inférieur au seuil de 0,05 °C.
- Avec le modèle Diffusion 2G, le temps de convergence typique est 5 000–15 000 s simulés pour un REP à pleine puissance, atteint en quelques dizaines de secondes réelles avec speedFactor = 100.
- T_max affiché à convergence représente la température maximale du combustible en °C, à comparer à la limite de fusion de l'UO_2 (1135°C).

9. Paramètres physiques par défaut

Paramètre	Valeur défaut	Unité	Source
Enrichissement REP	3,5 %	–	Industrie
T_entrée REP	286 °C	°C	EDF N4
T_sortie REP	326 °C	°C	EDF N4
Pression circuit primaire	155 bar	bar	REP 1300 MW
Puissance nominale	100 %	–	–
k UO_2	3,6	W/(m·K)	IAEA-TECDOC-1236
ρ UO_2	10 970	kg/m³	Idem
Cp UO_2	235	J/(kg·K)	Idem
α UO_2	$1{,}39\times10^{-6}$	m²/s	Calculé
φ0	$3{,}0\times10^{13}$	$n\cdot cm^{-2}\cdot s^{-1}$	REP typique
σf(U-235)	585	b	ENDF/B-VIII
E_fission	$3{,}2\times10^{-11}$	J	IAEA
ν (U-235)	2,43	–	ENDF/B-VIII
N_UO_2	$2{,}32\times10^{22}$	at/cm³	Calculé
nuSigF_th (REP 3,5%)	0,3230	cm^{-1}	ENDF + homogén.
D1 (groupe rapide)	1,35	cm	OECD/NEA
D2 (groupe thermique)	0,38	cm	OECD/NEA
ΣR1	0,033	cm^{-1}	Calculé
ΣR2	0,200	cm^{-1}	Calculé
ΣS12	0,0175	cm^{-1}	Calculé
Coefficient Doppler α_D	$-3{,}0\times10^{-5}$	K^{-1}	IAEA Safety Reports
Seuil de convergence	0,05	°C/frame	–
Tranches verticales défaut	8	–	–

10. Limites du modèle et perspectives

Limites actuelles

- **Géométrie 2D + 1D axiaux :** la neutronique est résolue en 2D (plan radial), et la thermique en 3D via l'extension axiale par tranches. Un couplage neutronique 3D complet (nécessitant un solveur multigroupe 3D) serait plus précis.
- **Sections efficaces paramétriques :** les XS 2G sont calculées par des corrélations simplifiées calibrées sur des données de référence. Un calcul exact nécessiterait un code de cellule (APOLLO2, Serpent-2) pour générer des bibliothèques de XS homogénéisées condensées.
- **Cinétique point :** le modèle ne traite pas les transitoires rapides (moins d'une seconde). Les neutrons retardés et la cinétique à 6 groupes de précurseurs ne sont pas implémentés.
- **Écoulement caloporteur monodimensionnel :** le modèle de caloporteur est unidimensionnel (montée de colonne). Les effets de mélange transversal, d'instabilités d'écoulement et de vide localisé ne sont pas représentés.
- **Feedback modérateur :** bien que la densité modérateur soit paramétrable, le couplage dynamique T_modérateur → ρ_modérateur → XS n'est pas résolu à chaque pas de temps.

Perspectives d'évolution

- Implémentation d'un solveur neutronique 3D (extension axiale du maillage FDM 2G actuel vers un maillage volumique 3G).
- Intégration des neutrons retardés pour la simulation de transitoires réalistes (insertion/extraction de barres, variations de puissance).
- Génération automatique des sections efficaces homogénéisées par couplage avec Serpent-2 (Monte Carlo) ou OpenMC.
- Modèle thermohydraulique 1D complet par canal (SUBCHANFLOW, COBRA-TF) en remplacement du modèle colonne actuel.
- Extension à des géométries hexagonales pour les RNR (maillage en nid d'abeille).
- Interface de chargement de fichiers de benchmark OECD/NEA (C5G7, BEAVRS) pour validation quantitative.

Références bibliographiques

[0] J. Bernard Énergie Nucléaire, tome 1 et 2 (2008) [1] J. Duderstadt, L. Hamilton, Nuclear Reactor Analysis, Wiley, 1976[2] P. Reuss, Neutron Physics, EDP Sciences, 2008[3] IAEA-TECDOC-1236, Thermohydraulic Relationships for Advanced Water Cooled Reactors, 2001[4] D. Brown et al., ENDF/B-VIII.0, Nuclear Data Sheets 148, 2018[5] T. Downar et al., PARCS v3.0 Theory Manual, Purdue University, 2012[6] LunarG, Vulkan SDK Documentation, vulkan.lunarg.com[7] raysan5, Raylib 5.0, github.com/raysan5/raylib